MANIPAL ACADEMY OF HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL INSTITUTE OF TECHNOLOGY

School of Computer Engineering

[Insert official MAHE / MIT Manipal emblem here]

CES 2112 Digital Systems Lab

(Open Ended Experiment Report)

on

TWO-CAR ELEVATOR COORDINATOR: A SYNCHRONOUS DIGITAL CONTROLLER IN VERILOG HDL

Submitted by

Team member name	Section	Registration / Roll No.
[Member 1 full name]	[Sec.]	[Reg. No.] / [Roll No.]
[Member 2 full name]	[Sec.]	[Reg. No.] / [Roll No.]
[Member 3 full name]	[Sec.]	[Reg. No.] / [Roll No.]
[Member 4 full name]	[Sec.]	[Reg. No.] / [Roll No.]
[Member 5 full name]	[Sec.]	[Reg. No.] / [Roll No.]

Under the guidance of:

Dr. Avik Bose, Assistant Professor

School of Computer Engineering, MIT Manipal

Date: 21/10/2026

TABLE OF CONTENTS

1. PROBLEM STATEMENT 3
System requirements 3
2. OBJECTIVES 3
2.1 Primary objective 3
2.2 Specific objectives 4
2.3 Scope and deliverables 4
3. SYSTEM DESIGN AND BLOCK DIAGRAM 4
3.1 Proposed architecture 4
3.2 Module description 5
3.3 Design decisions 6
4. VERILOG HDL IMPLEMENTATION 6
4.1 Design module 7
4.2 Code explanation 11
4.3 Testbench 12
5. SIMULATION, VERIFICATION AND RESULTS 12
5.1 Experimental setup 12
5.2 Test cases and expected outcomes 12
5.3 Simulation waveform 13
5.4 Results and discussion 14
6. INDIVIDUAL CONTRIBUTION 14
Contribution statement 15
7. CONCLUSION AND FUTURE SCOPE 15
7.1 Conclusion 15
7.2 Limitations and future scope 15
REFERENCES 15

1. PROBLEM STATEMENT

A building with more than one lift needs a **group controller**: a circuit that decides which car answers each button pressed on a landing. The decision has to be made continuously, while the cars are moving and new calls keep arriving. A poor controller sends both cars to the same floor, leaves one car idle while the other is overloaded, or forgets a request so that a passenger waits indefinitely [5].

Group controllers are usually software on a microcontroller or PLC, whose timing depends on the program and is hard to reason about cycle by cycle. The same function can instead be built as a small synchronous digital circuit in which every decision is held in a register and can be checked clock by clock in simulation.

This project designs, in synthesizable Verilog HDL [3], [4], a coordinator for **two elevator cars serving eight floors (F0 to F7)**. Hall calls (the UP and DOWN buttons on each landing) are stored and assigned, one per clock cycle, to the car that is nearer to the calling floor. Each car is a four-state finite state machine that collects its own cabin calls and the hall calls assigned to it, and visits them by continuing in its current direction while work remains in that direction. Travel time and door-open time are modelled with counters, so that behaviour is realistic in clock cycles.

Scope and assumptions. Inputs are 8-bit button vectors (bit k = floor k) that are high for one clock cycle per press and are synchronous to the system clock. Outputs are the floor, state, direction, timer and cabin calls of each car, the door-open signal, and the six hall-call registers that would drive the landing lamps. One floor takes 8 clock cycles and the door stays open for 10. Motor drives, load weighing, emergency and fire modes, button debouncing and physical FPGA implementation are outside the scope; the design is verified in simulation.

System requirements

TABLE I. SYSTEM REQUIREMENTS

ID	Requirement	Verification method
R1	Every hall call is latched until answered and is assigned to exactly one car; the nearer car wins and a tie goes to Car A.	Test cases T3 to T6; waveform (Fig. 3)
R2	Each car serves its cabin calls and assigned hall calls, keeping its direction while work remains ahead.	Test case T2; trace inspection
R3	Travel takes 8 clock cycles per floor and the door stays open for 10 clock cycles.	Waveform: timer counts 0 to 7 and 0 to 9
R4	Safety: a door is never open while a car moves; the floor register changes only in a MOVE state.	Test case T7, checked on every cycle
R5	Liveness: every hall call is answered within 600 clock cycles under random traffic.	Test case T8, five random seeds
R6	Synchronous reset clears every register; single clock, no combinational feedback loops.	Test case T1; code review

2. OBJECTIVES

2.1 Primary objective

To design and verify a synthesizable Verilog controller that assigns every hall call of an eight-floor building to the nearer of two elevator cars and drives both cars safely to all of their requests.

2.2 Specific objectives

- Implement the controller from standard combinational and sequential building blocks taught in the course: a 3-to-8 decoder, an 8-to-3 priority encoder, magnitude comparators, subtractors, multiplexers, registers, counters and a finite state machine [1], [2].
- Split the design into five design components (combinational building blocks, car FSM, hall-call dispatcher, top-level integration with testbench, and visualisation with co-simulation) so that each team member owns one component.
- Verify the design with a testbench driven by reproducible random stimulus, checking safety and liveness on every clock cycle.
- Cross-check the Verilog against an independent cycle-accurate reference model on every register and every cycle.
- Measure performance (waiting time and car utilisation) over several random traffic patterns.

2.3 Scope and deliverables

- Seven Verilog-2005 modules (`rtl/`): `decoder3to8`, `floor_masks`, `priority_encoder8`, `distance`, `elevator_car`, `hall_call_dispatcher` and the top level `elevator_coordinator`.
- Testbench `tb/tb_elevator_coordinator.v`, a seeded stimulus generator, and a one-command flow (`run.sh`) that compiles, simulates and runs all checks.
- VCD waveforms viewable in GTKWave [7], and an interactive web visualisation driven by the same register values.

3. SYSTEM DESIGN AND BLOCK DIAGRAM

The design is a single-clock synchronous system made of three blocks: one hall-call dispatcher and two identical elevator cars. All storage is in edge-triggered D flip-flops that update together on the rising edge of `clk`; a synchronous active-high reset `rst` clears them. Every request is held in an 8-bit **floor vector** in which bit k stands for floor k, so set operations on requests become single gates applied to all eight bits in parallel: OR is union, AND is intersection, and AND-NOT removes a served floor.

3.1 Proposed architecture

Fig. 1 shows the complete architecture. Hall buttons go only to the dispatcher; cabin buttons go straight to their own car. The dispatcher tells each car which hall calls belong to it (`up_asg0`, `dn_asg0`, `up_asg1`, `dn_asg1`); each car reports its current floor and, while its door is open, which hall calls it has just answered (`serve_up`, `serve_dn`). The cars never communicate directly, and the top-level module `elevator_coordinator` contains no logic of its own: it only instantiates and wires the three blocks.

Inside the dispatcher (left), the free calls (pending and not yet assigned) feed two priority encoders; a 2-to-1 multiplexer gives UP calls precedence; two distance circuits and a comparator choose the nearer car; and a decoder turns the chosen floor back into a one-hot vector that sets the `asgd` and `owner` bits. Inside each car (right), a decoder and a floor-mask table turn the 3-bit floor number into the vectors `here`, `above` and `below`, which are ANDed with the car's target floors and OR-reduced to give the three status signals used by the state machine. Car B is an identical copy of Car A.

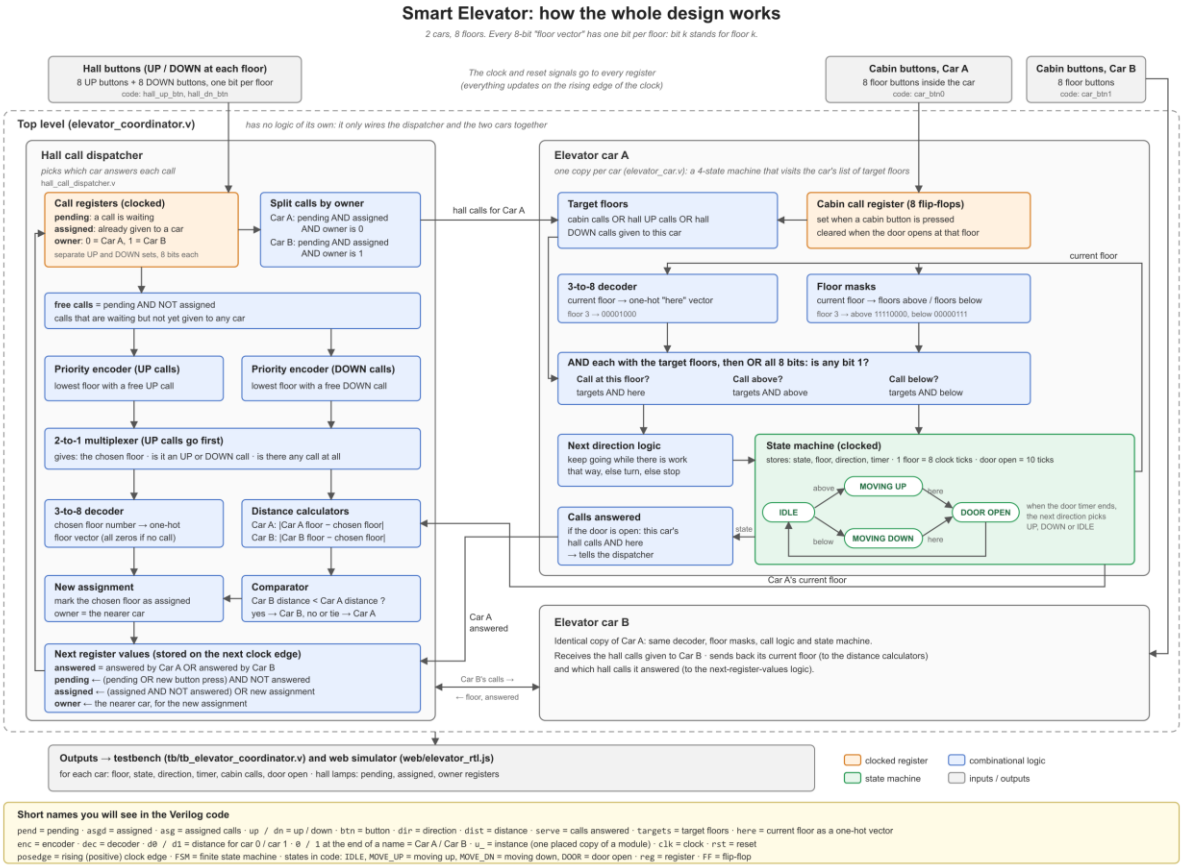


Fig. 1. Block diagram of the two-car elevator coordinator: hall-call dispatcher (left) and elevator car datapath with state machine (right).

3.2 Module description

TABLE II. MODULE DESCRIPTION

Module	Inputs	Outputs	Function
`elevator_coordinator` (top)	clk, rst, hall_up_btn, hall_dn_btn, car_btn0, car_btn1	Status of both cars; six hall-call registers	Structural top level; instantiates the dispatcher and two cars and wires them together.
`hall_call_dispatcher`	clk, rst, hall buttons, floor0, floor1, serve strobes of both cars	up/dn_pend, up/dn_asgd, up/dn_owner; calls assigned to each car	Stores hall calls, assigns one free call per cycle to the nearer car, clears calls when served.
`elevator_car` (x2)	clk, rst, car_btn, hall_up_asg, hall_dn_asg	floor, state, dir, timer, car_calls, next_dir, door_open, serve_up, serve_dn	Four-state FSM with a datapath that moves one car between floors and opens its door.
`priority_encoder8`	w[7:0]	y[2:0], valid	Index of the lowest 1 bit (lowest waiting call); valid = any bit set.
`decoder3to8`	en, w[2:0]	y[7:0]	Floor number to one-hot floor vector; all zeros when disabled.
`distance`	a[2:0], b[2:0]	d[2:0]	|a - b| using a comparator, two subtractors and a 2-to-1 multiplexer.
`floor_masks`	floor[2:0]	above[7:0], below[7:0]	Truth table of the floors above and below the car.

3.3 Design decisions

- **Single clock, synchronous reset.** Every register is written in an `always @(posedge clk)` block with `if (rst)` inside, so reset acts only on a clock edge and there is one clock domain.
- **Combinational vs. sequential split.** Decoders, masks, the priority encoders, distances, the comparator and the serve logic are combinational (`assign` and `always @(...)` with blocking `=`). Only 86 bits are stored (19 per car: state 2, floor 3, dir 2, timer 4 and car_calls 8; 48 in the dispatcher), all with non-blocking `<=`. The car's serve strobes depend on the dispatcher's registers, and the dispatcher's next state depends on the strobes; that loop passes through flip-flops, so there is no combinational feedback.
- **Binary state encoding.** The four car states use 2 bits (`IDLE`=00, `MOVE_UP`=01, `MOVE_DN`=10, `DOOR`=11); directions use `NONE`=00, `UP`=01, `DN`=10.
- **Parameterised timing.** `TRAVEL_TICKS` = 8 and `DOOR_TICKS` = 10 are module parameters; the 4-bit timer is a modulo counter with synchronous clear.
- **Assignment rule.** One free call is assigned per clock cycle, lowest floor first and UP before DOWN. A car needs 8 cycles to move one floor, so this is fast enough. The call goes to the car with the smaller distance (1); on a tie, Car A wins, which keeps the hardware deterministic:

$$\texttt{dist0} = |\,\texttt{floor0} - \texttt{pick_floor}\,|, \quad \texttt{dist1} = |\,\texttt{floor1} - \texttt{pick_floor}\,| \tag{1}$$

$$\texttt{pick_car1} = (\,\texttt{dist1} < \texttt{dist0}\,) \tag{2}$$

Here `floor0` and `floor1` are the current floors of Car A and Car B and `pick_floor` is the floor of the call chosen by the priority encoder; `pick_car1` = 1 assigns the call to Car B. Each car's behaviour is the state machine of Fig. 2. A car leaves `IDLE` for `DOOR` if it has a target on its floor, otherwise for the direction that has work. While moving it counts 8 ticks per floor; at each floor (`timer` = 0) it stops if there is a target there. After 10 door ticks it continues in the direction chosen by `next_dir`, which keeps the current direction while work remains ahead (the SCAN rule [5]).

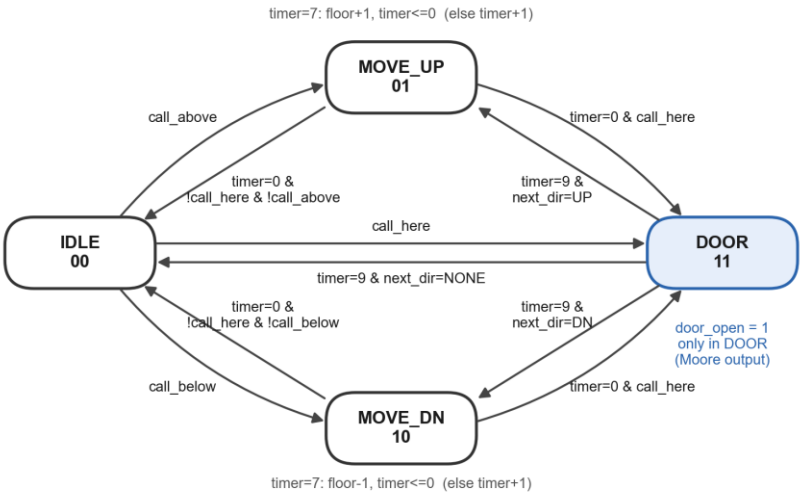


Fig. 2. State diagram of elevator_car (Moore output door_open = 1 only in DOOR).

4. VERILOG HDL IMPLEMENTATION

The design uses only Verilog-2005 constructs covered in the course: modules connected by position, `assign`, `always` blocks with `if`/`case`/`casex`, reduction operators, and non-blocking assignment for registers [3], [4]. Listings 1 to 7 give the complete synthesizable code. To save space, the explanatory comment blocks of the source files are omitted here; end-of-line comments are kept.

4.1 Design module

Listing 1: rtl/decoder3to8.v

```
module decoder3to8 (en, w, y);
    input        en;        // enable
    input  [2:0] w;         // floor number
    output reg [7:0] y;     // one-hot output
    always @(en or w)
    begin
        y = 8'b00000000;            // start with every output 0
        if (en == 1)
            case (w)
                0: y = 8'b00000001;
                1: y = 8'b00000010;
                2: y = 8'b00000100;
                3: y = 8'b00001000;
                4: y = 8'b00010000;
                5: y = 8'b00100000;
                6: y = 8'b01000000;
                7: y = 8'b10000000;
            endcase
    end
endmodule
```

Listing 2: rtl/floor_masks.v

```
module floor_masks (floor, above, below);
    input  [2:0] floor;     // the floor the car is at
    output reg [7:0] above; // 1 for every floor higher than 'floor'
    output reg [7:0] below; // 1 for every floor lower  than 'floor'
    always @(floor)
        case (floor)
            0: begin above = 8'b11111110; below = 8'b00000000; end
            1: begin above = 8'b11111100; below = 8'b00000001; end
            2: begin above = 8'b11111000; below = 8'b00000011; end
            3: begin above = 8'b11110000; below = 8'b00000111; end
            4: begin above = 8'b11100000; below = 8'b00001111; end
            5: begin above = 8'b11000000; below = 8'b00011111; end
            6: begin above = 8'b10000000; below = 8'b00111111; end
            7: begin above = 8'b00000000; below = 8'b01111111; end
        endcase
endmodule
```

Listing 3: rtl/priority_encoder8.v

```
module priority_encoder8 (w, y, valid);
    input  [7:0] w;
    output reg [2:0] y;
    output reg       valid;
    always @(w)
    begin
        valid = 1;
        casex (w)
            8'bxxxxxxx1: y = 0;
            8'bxxxxxx10: y = 1;
            8'bxxxxx100: y = 2;
            8'bxxxx1000: y = 3;
            8'bxxx10000: y = 4;
            8'bxx100000: y = 5;
            8'bx1000000: y = 6;
            8'b10000000: y = 7;
            default: begin y = 0; valid = 0; end   // w = 00000000
        endcase
    end
endmodule
```

Listing 4: rtl/distance.v

```
module distance (a, b, d);
    input  [2:0] a, b;
    output [2:0] d;
    assign d = (a > b) ? (a - b) : (b - a);
endmodule
```

Listing 5: rtl/elevator_car.v

```
module elevator_car (clk, rst, car_btn, hall_up_asg, hall_dn_asg,
                     floor, state, dir, timer, car_calls,
                     next_dir, door_open, serve_up, serve_dn);
```

```
    parameter TRAVEL_TICKS = 8;     // clock cycles to move one floor
    parameter DOOR_TICKS   = 10;    // clock cycles the door stays open
    parameter IDLE    = 2'b00;
    parameter MOVE_UP = 2'b01;
    parameter MOVE_DN = 2'b10;
    parameter DOOR    = 2'b11;
    parameter NONE = 2'b00;
    parameter UP   = 2'b01;
    parameter DN   = 2'b10;
    input        clk;            // clock
    input        rst;            // synchronous reset, active high
    input  [7:0] car_btn;        // cabin buttons (bit k = floor k), 1 for one clock
    input  [7:0] hall_up_asg;    // hall UP   calls the dispatcher gave to this car
    input  [7:0] hall_dn_asg;    // hall DOWN calls the dispatcher gave to this car
    output reg [2:0] floor;      // floor the car is at (or last passed)
    output reg [1:0] state;      // IDLE / MOVE_UP / MOVE_DN / DOOR
    output reg [1:0] dir;        // direction the car is travelling in
    output reg [3:0] timer;      // counts clock cycles while moving / door open
    output reg [7:0] car_calls;  // cabin buttons pressed and not yet served
    output reg [1:0] next_dir;   // where the car will go next (combinational)
    output       door_open;      // 1 while the door is open
    output [7:0] serve_up;       // hall UP   call answered at this floor right now
    output [7:0] serve_dn;       // hall DOWN call answered at this floor right now
    wire [7:0] here;             // one-hot: the floor the car is on
    wire [7:0] above;            // mask of floors above the car
    wire [7:0] below;            // mask of floors below the car
    wire [7:0] targets;          // every floor this car must visit
    wire       call_here;        // is there a target on this floor?
    wire       call_above;       // is there a target above?
    wire       call_below;       // is there a target below?
    decoder3to8 dec_here (1'b1, floor, here);
    floor_masks masks (floor, above, below);
    assign targets = car_calls | hall_up_asg | hall_dn_asg;
    assign call_here  = |(targets & here);
    assign call_above = |(targets & above);
    assign call_below = |(targets & below);
    always @(dir or call_above or call_below)
    begin
        if (dir == UP && call_above)
            next_dir = UP;
        else if (dir == DN && call_below)
            next_dir = DN;
        else if (call_above)
            next_dir = UP;
        else if (call_below)
            next_dir = DN;
        else
            next_dir = NONE;
    end
    assign door_open = (state == DOOR);
    assign serve_up = door_open ? (hall_up_asg & here) : 8'b00000000;
    assign serve_dn = door_open ? (hall_dn_asg & here) : 8'b00000000;
    always @(posedge clk)
    begin
        if (rst)
            car_calls <= 8'b00000000;
        else if (state == DOOR)
            car_calls <= (car_calls | car_btn) & ~here;
        else
            car_calls <= car_calls | car_btn;
    end
    always @(posedge clk)
    begin
        if (rst)
        begin
            state <= IDLE;
            floor <= 0;
            dir   <= NONE;
            timer <= 0;
        end
        else
            case (state)
            IDLE:
            begin
                timer <= 0;
                if (call_here)
                    state <= DOOR;                          // rule 1
                else if (call_above)
                begin
```

```
                    state <= MOVE_UP;
                    dir   <= UP;
                end
                else if (call_below)
                begin
                    state <= MOVE_DN;
                    dir   <= DN;
                end
                else
                    dir <= NONE;                            // rule 4
            end
            MOVE_UP:
            begin
                if (timer == 0 && call_here)
                    state <= DOOR;                          // rule 1: stop here
                else if (timer == 0 && !call_above)
                begin
                    state <= IDLE;                          // safety: nothing above
                    dir   <= NONE;
                end
                else if (timer == TRAVEL_TICKS - 1)
                begin
                    floor <= floor + 1;                     // reached the next floor
                    timer <= 0;
                end
                else
                    timer <= timer + 1;                     // still between floors
            end
            MOVE_DN:
            begin
                if (timer == 0 && call_here)
                    state <= DOOR;
                else if (timer == 0 && !call_below)
                begin
                    state <= IDLE;
                    dir   <= NONE;
                end
                else if (timer == TRAVEL_TICKS - 1)
                begin
                    floor <= floor - 1;
                    timer <= 0;
                end
                else
                    timer <= timer + 1;
            end
            DOOR:
            begin
                if (timer == DOOR_TICKS - 1)
                begin
                    timer <= 0;                             // door closes
                    if (next_dir == UP)                     // rules 2 and 3
                    begin
                        state <= MOVE_UP;
                        dir   <= UP;
                    end
                    else if (next_dir == DN)
                    begin
                        state <= MOVE_DN;
                        dir   <= DN;
                    end
                    else
                    begin
                        state <= IDLE;
                        dir   <= NONE;
                    end
                end
                else
                    timer <= timer + 1;
            end
            endcase
    end
endmodule
```

Listing 6: `rtl/hall_call_dispatcher.v`

```
module hall_call_dispatcher (clk, rst, hall_up_btn, hall_dn_btn,
                             floor0, floor1,
                             serve_up0, serve_dn0, serve_up1, serve_dn1,
                             up_pend, up_asgd, up_owner,
```

```
                            dn_pend, dn_asgd, dn_owner,
                            up_asg0, dn_asg0, up_asg1, dn_asg1);
    input        clk, rst;
    input  [7:0] hall_up_btn;    // hall UP   buttons (1 for one clock when pressed)
    input  [7:0] hall_dn_btn;    // hall DOWN buttons
    input  [2:0] floor0;         // where Car A is
    input  [2:0] floor1;         // where Car B is
    input  [7:0] serve_up0, serve_dn0;   // calls Car A answered this cycle
    input  [7:0] serve_up1, serve_dn1;   // calls Car B answered this cycle
    output reg [7:0] up_pend, up_asgd, up_owner;   // UP call registers
    output reg [7:0] dn_pend, dn_asgd, dn_owner;   // DOWN call registers
    output [7:0] up_asg0, dn_asg0;   // calls given to Car A
    output [7:0] up_asg1, dn_asg1;   // calls given to Car B
    wire [7:0] free_up, free_dn;     // waiting but not yet given to a car
    wire [2:0] up_floor, dn_floor;   // lowest free call of each kind
    wire       up_valid, dn_valid;   // is there a free call of that kind?
    wire       pick_valid;           // is there any call to hand out?
    wire       pick_up;              // 1 = it is an UP call, 0 = a DOWN call
    wire [2:0] pick_floor;           // floor of the call being handed out
    wire [7:0] pick_vec;             // the same floor as a one-hot vector
    wire [2:0] dist0, dist1;         // how far each car is from pick_floor
    wire       pick_car1;            // 1 = give it to Car B, 0 = Car A
    wire [7:0] new_up, new_dn;       // the call handed out this cycle
    wire [7:0] served_up, served_dn; // calls answered by either car
    assign up_asg0 = up_pend & up_asgd & ~up_owner;
    assign dn_asg0 = dn_pend & dn_asgd & ~dn_owner;
    assign up_asg1 = up_pend & up_asgd &  up_owner;
    assign dn_asg1 = dn_pend & dn_asgd &  dn_owner;
    assign free_up = up_pend & ~up_asgd;
    assign free_dn = dn_pend & ~dn_asgd;
    priority_encoder8 enc_up (free_up, up_floor, up_valid);
    priority_encoder8 enc_dn (free_dn, dn_floor, dn_valid);
    assign pick_valid = up_valid | dn_valid;
    assign pick_up    = up_valid;                          // UP calls go first
    assign pick_floor = up_valid ? up_floor : dn_floor;    // 2-to-1 multiplexer
    distance d0 (floor0, pick_floor, dist0);
    distance d1 (floor1, pick_floor, dist1);
    assign pick_car1 = (dist1 < dist0);     // tie -> Car A
    decoder3to8 dec_pick (pick_valid, pick_floor, pick_vec);
    assign new_up = pick_up ? pick_vec : 8'b00000000;
    assign new_dn = pick_up ? 8'b00000000 : pick_vec;
    assign served_up = serve_up0 | serve_up1;
    assign served_dn = serve_dn0 | serve_dn1;
    always @(posedge clk)
    begin
        if (rst)
        begin
            up_pend <= 0;  up_asgd <= 0;  up_owner <= 0;
            dn_pend <= 0;  dn_asgd <= 0;  dn_owner <= 0;
        end
        else
        begin
            up_pend <= (up_pend | hall_up_btn) & ~served_up;
            dn_pend <= (dn_pend | hall_dn_btn) & ~served_dn;
            up_asgd <= (up_asgd & ~served_up) | new_up;
            dn_asgd <= (dn_asgd & ~served_dn) | new_dn;
            if (pick_car1)
            begin
                up_owner <= up_owner | new_up;      // new call belongs to Car B
                dn_owner <= dn_owner | new_dn;
            end
            else
            begin
                up_owner <= up_owner & ~new_up;     // new call belongs to Car A
                dn_owner <= dn_owner & ~new_dn;
            end
        end
    end
endmodule
```

Listing 7: `rtl/elevator_coordinator.v`

```
module elevator_coordinator (clk, rst,
                            hall_up_btn, hall_dn_btn, car_btn0, car_btn1,
                            floor0, state0, dir0, timer0, calls0, door0,
                            floor1, state1, dir1, timer1, calls1, door1,
                            up_pend, up_asgd, up_owner,
                            dn_pend, dn_asgd, dn_owner);
    input        clk, rst;
```

```
    input  [7:0] hall_up_btn, hall_dn_btn;   // landing buttons
    input  [7:0] car_btn0, car_btn1;         // buttons inside Car A / Car B
    output [2:0] floor0, floor1;
    output [1:0] state0, state1;
    output [1:0] dir0,   dir1;
    output [3:0] timer0, timer1;
    output [7:0] calls0, calls1;
    output       door0,  door1;
    output [7:0] up_pend, up_asgd, up_owner;
    output [7:0] dn_pend, dn_asgd, dn_owner;
    wire [7:0] up_asg0, dn_asg0, up_asg1, dn_asg1;          // dispatcher -> cars
    wire [7:0] serve_up0, serve_dn0, serve_up1, serve_dn1;  // cars -> dispatcher
    wire [1:0] next_dir0, next_dir1;                        // (shown on the website)
    hall_call_dispatcher u_dispatch (
        clk, rst,
        hall_up_btn, hall_dn_btn,
        floor0, floor1,
        serve_up0, serve_dn0, serve_up1, serve_dn1,
        up_pend, up_asgd, up_owner,
        dn_pend, dn_asgd, dn_owner,
        up_asg0, dn_asg0, up_asg1, dn_asg1);
    elevator_car u_car0 (
        clk, rst, car_btn0, up_asg0, dn_asg0,
        floor0, state0, dir0, timer0, calls0,          // registers
        next_dir0, door0, serve_up0, serve_dn0);       // combinational outputs
    elevator_car u_car1 (
        clk, rst, car_btn1, up_asg1, dn_asg1,
        floor1, state1, dir1, timer1, calls1,
        next_dir1, door1, serve_up1, serve_dn1);
endmodule
```

4.2 Code explanation

Building blocks. `decoder3to8` turns a 3-bit floor number into a one-hot floor vector and is used twice: for the car's current floor (`here`) and for the floor picked by the dispatcher (`pick_vec`, disabled when there is no call). `floor_masks` is a lookup table, written with `case`, that returns the floors above and below the car. `priority_encoder8` uses `casex` with don't-care bits so that the lowest set bit wins. `distance` computes |a − b| without signed arithmetic: the comparator `a > b` selects which subtraction to output.

elevator_car. Part 1 is combinational. `targets` is the union of the cabin calls and the hall calls assigned to this car; ANDing it with `here`, `above` and `below` and OR-reducing gives `call_here`, `call_above` and `call_below`. `next_dir` is a priority multiplexer: keep the current direction if there is work that way, otherwise turn towards any work, otherwise `NONE`. `door_open` is a Moore output (`state == DOOR`), and `serve_up`/`serve_dn` report the assigned hall calls on this floor while the door is open. Part 2 is the cabin-call register: set by a button press (OR) and cleared for the current floor while the door is open (AND-NOT). Part 3 is the state machine, which also updates the `floor` up/down counter, the `dir` register and the `timer` counter. `floor` changes only inside `MOVE_UP`/`MOVE_DN`, and the door is open only in `DOOR`, so safety (R4) holds by construction.

hall_call_dispatcher. For each direction, `pend` is set by a button and cleared when either car serves the call; `asgd` is set when the call is handed out and cleared when served; `owner` records the chosen car. A call is free when `pend & ~asgd`. The vectors given to each car are pure AND logic: Car A gets `pend & asgd & ~owner` and Car B gets `pend & asgd & owner`. A call cannot be picked and served in the same cycle, because picking needs `asgd` = 0 and serving needs `asgd` = 1.

elevator_coordinator. Structural only: it instantiates `u_dispatch`, `u_car0` and `u_car1` and connects them by position. `next_dir0`/`next_dir1` are brought out only so that the visualisation can display them.

4.3 Testbench

The testbench (Listing 8) follows the lab layout: it `includes` the design files, declares a `reg` for each input and a `wire` for each output, instantiates the design, and generates a 10 ns clock with `forever`. Reset is held for the first two rising edges (5 ns and 15 ns) and released at 16 ns. The button presses come from `build/stimulus.v`, written by `scripts/gen_stimulus.js` with a seeded pseudo-random generator (mulberry32), so every run can be reproduced exactly from its seed. Each line applies one clock cycle of inputs and waits 10 ns, so inputs change 1 ns after each rising edge and are stable for 9 ns before the next one. Per cycle there is a 3.5 % chance of an UP call, 3.5 % of a DOWN call and 1.5 % of a cabin call in each car; impossible calls (UP at F7, DOWN at F0) are never generated.

Listing 8: `tb/tb_elevator_coordinator.v` (essential part; port declarations and `include` lines omitted)

```
elevator_coordinator dut (clk, rst, hall_up_btn, hall_dn_btn, car_btn0, car_btn1,
                          floor0, state0, dir0, timer0, calls0, door0,
                          floor1, state1, dir1, timer1, calls1, door1,
                          up_pend, up_asgd, up_owner, dn_pend, dn_asgd, dn_owner);
initial begin                          // clock: period 10 ns
    clk = 0;
    forever #5 clk = ~clk;
end
initial begin
    $dumpfile("build/elevator.vcd");
    $dumpvars(0, tb_elevator_coordinator);
    rst = 1;                           // reset for the first two clock edges
    hall_up_btn = 0; hall_dn_btn = 0; car_btn0 = 0; car_btn1 = 0;
    #16;
    rst = 0;
    `include "stimulus.v"          // one line per clock cycle, e.g.
    // hall_up_btn = 8'h00; hall_dn_btn = 8'h04; car_btn0 = 8'h20; car_btn1 = 8'h00; #10;
    $display("Test complete");
    $finish;
end
```

Pass/fail is decided automatically from the VCD dump: `scripts/verify.js` checks liveness and compares all 16 output registers with a reference model on every cycle, and `scripts/vcd2js.js` compares 154 named signals, including internal nets such as `next_dir`, `pick_floor` and `pick_car1`.

5. SIMULATION, VERIFICATION AND RESULTS

5.1 Experimental setup

TABLE III. EXPERIMENTAL SETUP

Item	Details
HDL / language	Verilog-2005 (IEEE Std 1364-2005 [4]), compiled with `-g2005 -Wall`
Simulator / version	Icarus Verilog 14.0 (devel) [6], `iverilog` + `vvp`; VCD waveforms, viewable in GTKWave [7]
Synthesis tool / device	N/A (simulation only)
Stimulus / checking	Seeded random traffic, 4,000 cycles per run (seeds 1, 7, 42, 99, 2026); checks in Node.js v24
Test cases	9 checks (T1 to T9): directed cases found in the runs, plus whole-run safety, liveness and equivalence

5.2 Test cases and expected outcomes

Times are simulation times in the seed-2026 run; an input is applied 1 ns after a rising edge and registered on the next edge.

TABLE IV. TEST CASES

Test ID	Input / condition	Expected output	Observed result
T1	Reset held for the first two rising edges	All registers 0; both cars IDLE at F0 with doors closed	All 16 output registers 0 until the first press. Pass
T2	Cabin button F1 in Car B at 56 ns, car idle at F0	Car B moves up, reaches F1 after 8 cycles, opens door for 10 cycles	MOVE_UP at 75 ns, floor1 = 1 at 155 ns, DOOR 165 to 265 ns, then IDLE. Pass
T3	UP call at F1 at 176 ns while Car B's door is open at F1	Assigned to Car B (distance 0) and answered at once	Pending at 185 ns, assigned to B at 195 ns, cleared at 205 ns. Pass
T4	DOWN call at F2 at 506 ns; Car A at F0, Car B at F1	Nearer car (B, d1 = 1 < d0 = 2) gets the call	dn_owner bit 2 = 1 at 525 ns; Car B opens at F2 at 625 ns. Pass
T5	Tie: DOWN call at F7 at 3376 ns; both cars at F3	d0 = d1 = 4, so Car A gets the call	pick_car1 = 0; dn_owner bit 7 = 0 at 3395 ns. Pass
T6	UP F5 and DOWN F7 pressed in the same cycle (38956 ns)	One call per cycle, UP first	UP F5 assigned at 38975 ns, DOWN F7 at 38985 ns. Pass
T7	Safety, every cycle, both cars	door_open only in DOOR; floor changes only in MOVE states	0 violations in 8,000 car-cycles. Pass
T8	Liveness, five seeds	Every hall call answered within 600 cycles	1,033 calls answered; longest wait 193 cycles. Pass
T9	Co-simulation against the reference model	Identical values on every cycle	16 registers and 154 signals, every cycle, 5 seeds: 0 mismatches. Pass

5.3 Simulation waveform

Fig. 3 is drawn from the VCD dump of the seed-2026 run and shows test case T4. The numbered markers are: (1) at 506 ns a DOWN call at F2 and a cabin call for F5 in Car A arrive; (2) at the 515 ns edge the call is latched in `dn_pend` and the priority encoder outputs `pick_floor` = 2 with `dist0` = 2 and `dist1` = 1, so `pick_car1` = 1; (3) at 525 ns `dn_asgd` and `dn_owner` bit 2 are set (owner Car B), and Car A starts moving up towards its cabin call; (4) at 615 ns Car B's timer wraps and `floor1` becomes 2; (5) at 625 ns Car B enters `DOOR`, `door1` rises and `serve_dn1` = 04; (6) at 635 ns the call is cleared. The door then stays open for exactly 10 cycles (`timer1` 0 to 9).

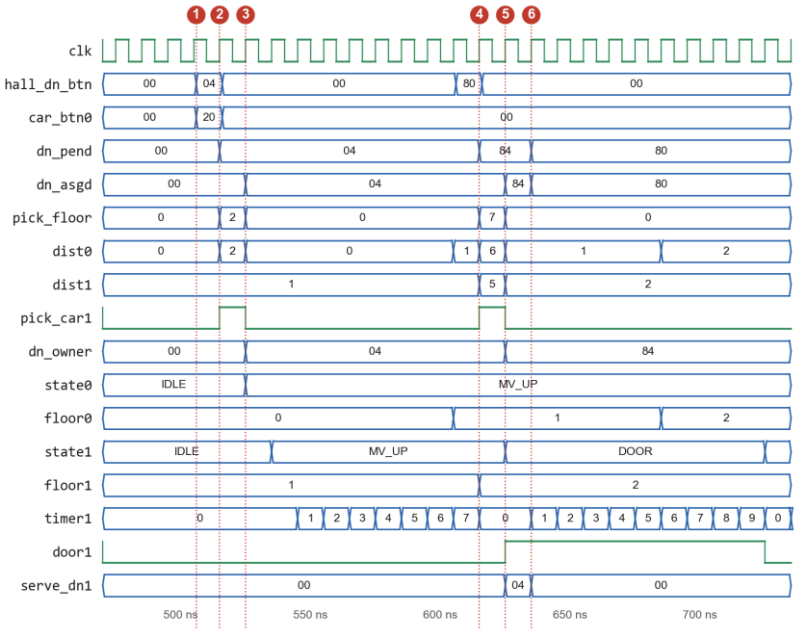


Fig. 3. Simulation waveform for test case T4 (seed 2026, 470 to 735 ns).

5.4 Results and discussion

All nine test cases and all whole-run checks passed: the observed outputs match the expected outputs in every case. Table V summarises performance over the five seeds. Wait is the number of cycles a hall call spends in a `pend` register; utilisation is the share of the 8,000 car-cycles per run spent in each state.

TABLE V. PERFORMANCE OVER FIVE RANDOM SEEDS (4,000 CYCLES EACH)

Seed	Calls answered	Mean wait (cycles)	Median wait (cycles)	Longest wait (cycles)	Moving (%)	Door open (%)	Idle (%)
1	222	37.2	23	165	51.4	31.8	16.9
7	205	52.3	38	186	53.3	32.8	14.0
42	192	39.5	27	193	46.9	30.6	22.5
99	213	37.5	23	159	50.3	33.3	16.5
2026	201	42.4	27	168	51.1	32.4	16.5

Every call was answered, with a longest wait of 193 cycles against the 600-cycle bound. The median is always below the mean: most calls are answered within about three floors of travel, while a few that arrive just after a car has passed must wait for the next sweep. Both cars are busy for 78 to 86 % of the time under this load (about one hall call every 14 cycles plus cabin calls). No timing or area figures are reported because the design was not synthesized.

6. INDIVIDUAL CONTRIBUTION

The project was divided into five design components so that each member owned one component end to end (design, code, unit checks and the matching part of this report).

TABLE VI. INDIVIDUAL CONTRIBUTIONS

Team member	Section / Reg. No. / Roll No.	Individual tasks performed	Evidence / deliverables
[Member 1]	[Sec., Reg. No., Roll No.]	Component 1: combinational building blocks. Designed the 3-to-8 decoder, floor-mask table, 8-to-3 priority encoder and distance circuit; checked each truth table.	`decoder3to8.v`, `floor_masks.v`, `priority_encoder8.v`, `distance.v`; Table II
[Member 2]	[Sec., Reg. No., Roll No.]	Component 2: elevator car. Designed the four-state FSM, direction logic, cabin-call register, timer and floor counters; drew the state diagram.	`elevator_car.v`; Fig. 2; Section 4.2
[Member 3]	[Sec., Reg. No., Roll No.]	Component 3: hall-call dispatcher. Designed the six call registers, free-call selection, nearest-car comparison and tie rule.	`hall_call_dispatcher.v`; Fig. 1; equations (1) and (2)
[Member 4]	[Sec., Reg. No., Roll No.]	Component 4: integration and verification. Wrote the top level, testbench, stimulus generator and run script; ran the five-seed regression and collected results.	`elevator_coordinator.v`, `tb_elevator_coordinator.v`, `run.sh`; Tables IV and V; Fig. 3
[Member 5]	[Sec., Reg. No., Roll No.]	Component 5: reference model and visualisation. Wrote the cycle-accurate JavaScript model, the co-simulation checks and the web visualisation with waveform viewer.	`web/elevator_rtl.js`, `verify.js`, `vcd2js.js`, `web/`; test case T9

Contribution statement

The interfaces between components (the floor-vector convention and the signals in Fig. 1) were agreed jointly at the start, so the components could be developed in parallel. Integration, debugging and the final regression were done together, led by the integration owner. Each member wrote the report section for their own component; the problem statement, objectives and conclusion were written jointly.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project delivers a working two-car, eight-floor elevator coordinator in 86 flip-flops of synchronous logic, built from decoders, a priority encoder, comparators, subtractors, multiplexers, registers, counters and a finite state machine. All objectives were met in simulation: every requirement R1 to R6 was checked, all nine test cases passed, all 1,033 hall calls in five random runs were answered (longest wait 193 cycles), no safety violation occurred, and the Verilog matched an independent reference model on every register and cycle.

7.2 Limitations and future scope

- **Distance-only dispatch.** The nearer car wins even if it is moving away from the call; adding direction and current load to the cost would reduce waiting time.
- **Direction of service.** A car answers any assigned hall call at its floor when the door opens, even if it then leaves in the other direction; serving an UP call only when the car will go up would be more realistic.
- **Assignments are final.** A call stays with its car even if the other car later becomes better placed; periodic re-assignment of unreached calls is a natural extension.
- **Not synthesized.** Synthesis to an FPGA (for example with Yosys or Vivado) would give measured LUT and flip-flop counts and the maximum clock frequency, and allow testing on a development board with synchronized, debounced button inputs.

REFERENCES

[1] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with Verilog Design, 3rd ed. New York, NY, USA: McGraw-Hill, 2014.

[2] M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed. Hoboken, NJ, USA: Pearson, 2018.

[3] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[4] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[5] G. Barney and L. Al-Sharif, Elevator Traffic Handbook: Theory and Practice, 2nd ed. Abingdon, U.K.: Routledge, 2016.

[6] S. Williams, “Icarus Verilog.” Accessed: Oct. 20, 2026. [Online]. Available: https://steveicarus.github.io/iverilog/

[7] T. Bybell, “GTKWave.” Accessed: Oct. 20, 2026. [Online]. Available: https://gtkwave.sourceforge.net/